CS 558: Computer Systems Lab
(January-May 2026)

Assignment – 4

Submission deadline: 11:55 PM, Wednesday, 26th March 2026

This assignment requires you to implement the scenarios described in this document using the **C/C++ programming language**.

Guidelines

- Each group must **implement all the questions** in the assignment.
- If you find that any part of a problem description is **missing, unclear, or incorrect**, clearly **state the assumptions** you made while providing your solution.
- Submit all **source code files of the application** as a **compressed file**. The name of the compressed file should correspond to your **group number**, for example:
  - `Group_4.zip`
  - `Group_4.rar`
  - `Group_4.tar.gz`
- Only **one member from each group** is required to submit the assignment by uploading the compressed file through the following form: https://forms.gle/kUhQdw5xbgg7jCVQ8
- The assignment will be **evaluated during the lab session**, either offline or through **viva voce**. During the evaluation, you will be required to **explain your solutions to the evaluator**.
- A **plagiarism detection tool** will be used to verify the originality of submissions. Any instance of **academic misconduct or unfair means** will result in **negative marks equal to the maximum marks allocated for the assignment**.

Question 1

In this problem, you are required to use **ordinary pipes** to implement an **inter-process communication (IPC)** mechanism for message passing between processes.

Assume that there are **two directories**, `d1` and `d2`, each containing multiple files. Every file stores a **short string of characters** as its content. You must design a program in which a **parent process creates two child processes**, and each child process is responsible for examining one directory.

Requirements

- **Child Process 1** will handle directory `d1`, while **Child Process 2** will handle directory `d2`.
- Each child process must:
  - Read the files present in its assigned directory.
  - Create a **list containing the file names and their corresponding contents**.
- After preparing the list, **each child process should send its list to the other child process using a pipe**.
- If a child process encounters an **error while reading any file in its directory**, it must immediately send an **error message along with the updates collected so far** to the other child process using the message passing mechanism.
- After receiving the list from the other child:
  - **Child 2** should create the files listed by **Child 1** inside directory `d2` and populate them with the provided contents.
  - **Child 1** should similarly create the files listed by **Child 2** inside directory `d1` and write the corresponding contents.
- After the program finishes executing, the contents of **directories `d1` and `d2` should become identical**.

Question 2

Write a **C/C++ program** that simulates a **basic online examination system** using **message passing for inter-process communication (IPC)**.

The program should first ask the user to input:

- The **number of students** appearing for the exam.
- The **number of questions** in the exam.

After receiving the input, the **parent process** should create a **separate child process for each student**. The parent will use **message queues** to send the exam questions to each child process.

Requirements

- Each **child process (representing a student)** should:
  - Receive the exam questions from the parent through a message queue.
  - Generate **random answers** for the questions.
  - Send the answers back to the parent process using **another message queue**.
- The **parent process** should:
  - Collect the answers submitted by all child processes.
  - **Evaluate and grade** each student's exam.
- After grading, the program should:
  - Display the **individual grade of each student**.
  - Show the **overall grade distribution** for the entire exam.
- Once the grading process is complete, the parent process must **wait for all child processes to terminate**, using a **message queue–based synchronization mechanism**.
- If any **child process fails to respond or terminates unexpectedly**, the program should:
  - Display an appropriate **warning message**.
  - Take suitable action to **handle the failure scenario**.

Question 3

In this problem, you need to implement a **multi-threaded event reservation system** for the **Nehru Centre** using the **pthread API** in C/C++. The system simulates concurrent user queries for booking, cancelling, and checking seat availability for multiple events.

Assume that:

- There are **e events**.
- The auditorium has a **capacity of c seats** for each event.

The reservation system must support the following **three types of queries**:

1. **Inquiry Query** – Check the number of available seats for a specific event.
2. **Booking Query** – Book **k tickets** for a specific event.
3. **Cancellation Query** – Cancel a previously booked ticket.

To avoid system overload, the server enforces a limit called **MAX**, which represents the **maximum number of queries that can be active at the same time**.

In addition, the system must ensure **data consistency** when multiple threads attempt to read or modify reservation data for the same event.

Program Behavior

Initialization

The **main (master) thread** performs the following tasks:

1. Creates a list of **e events**.
2. Initializes the **available seats for each event to c**.
3. Creates **s worker threads**.

Each worker thread repeatedly generates **random queries** and executes them concurrently.

Query Generation

Each worker thread periodically generates queries with randomly chosen parameters:

- **Query type** (Inquiry / Booking / Cancellation)
- **Event number** (for inquiry or booking queries)
- **Number of tickets k** for booking queries (random value between **5 and 10**)
- **Ticket to cancel** (selected randomly from the thread's private booking list)

During each query:

- The thread sleeps for a **random short interval** to simulate processing time (such as transaction delays).
- Between two consecutive queries, the thread also sleeps for a **random short duration**.

Limiting Concurrent Queries

At any time, **no more than MAX queries may be active**.

If a thread attempts to start a query while **MAX queries are already active**, it must **block** until one of the active queries finishes.

This restriction must be implemented using **condition variables**.
A waiting thread should remain blocked until another thread signals that the server load has decreased.

Consistency Rules for Read and Write Queries

To maintain database consistency:

Write Queries (Booking and Cancellation)

- **Multiple write operations cannot occur simultaneously on the same event.**
- A **write operation cannot occur while a read query is active** for that event.
- However, **writes on different events may proceed concurrently**.

Read Queries (Inquiry)

- Multiple **read operations can run simultaneously** on the same event.
- A read operation must not proceed if a **write operation is currently active** for that event.

Shared Query Table

Instead of creating a mutex for every event (which may be inefficient when **e is large**), maintain a **shared table with MAX entries**.

Each entry in the table contains:

- **Event number**
- **Query type**
- **Thread number that initiated the query**

A **blank entry** is represented by setting the **event number to –1**.

Query Admission Rules

- A **read query** can proceed only if the corresponding event **does not appear in the table with a write query**.
- A **write query** can proceed only if the corresponding event **does not appear in the table with either a read or write query**.

Access to the shared table must be protected by a **mutex**.

When a thread successfully starts a query:

1. It **fills a blank entry** in the table with the query details.
2. After completing the query, it **removes its entry** from the table.

If a query **fails due to consistency constraints**, the thread **does not block**. Instead, it sleeps briefly and then attempts another query.

System Shutdown

The system runs for a predetermined duration **T**.

After time **T** expires:

1. All **worker threads terminate one by one**.
2. The **master thread waits for all worker threads** to finish using an appropriate **barrier synchronization mechanism**.
3. Once all threads have terminated, the master thread prints the **final reservation status for all events** and exits.

Parameter Values

Your program should handle the following **typical parameter values**:

Parameter	Description	Typical Value
e	Number of events	100
c	Capacity of each event	500
k	Tickets booked in one booking query	Random integer between **5 and 10**
s	Number of worker threads	20
MAX	Maximum active queries	5
T	Total execution time	**1–10 minutes**

Additional Requirements

- Each worker thread should maintain a **private list of bookings** it has made.
- A **cancellation request** must be randomly selected from this private list.
- Threads must print **diagnostic messages**, including:
    - Query type and parameters
    - Thread ID
    - Waiting and signaling events
    - Query results
    - Timeouts and thread termination